

LAPORAN PRAKTIKUM DAN TUGAS

STRUKTUR DATA

“DOUBLE LINKED LIST”

DISUSUN OLEH:

Aliyatar Rafi Ahmad

2511533031

DOSEN PENGAMPU:

Dr. Wahyudi, S.T, M.T

ASISTEN PRAKTIKUM:

Sherly Sukmadira



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum Struktur Data dan Algoritma Pekan ke-6 tentang *Double Linked List* ini dapat diselesaikan dengan baik. Laporan ini dibuat sebagai bentuk dokumentasi dan pemahaman terhadap materi praktikum yang telah dilakukan.

Pada praktikum ini dipelajari konsep dasar *Double Linked List* beserta implementasinya menggunakan bahasa pemrograman Java. Melalui praktikum ini, mahasiswa dapat memahami cara kerja node yang memiliki dua pointer, yaitu pointer ke node berikutnya dan node sebelumnya, sehingga proses penelusuran data dapat dilakukan dua arah.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan agar laporan ini dapat menjadi lebih baik di masa mendatang. Semoga laporan ini dapat bermanfaat bagi pembaca dan menambah pemahaman mengenai struktur data *Double Linked List*.

Padang, 13 Mei 2026

Aliyatar Rafi Ahmad

DAFTAR ISI

KATA PENGANTAR.....	2
BAB I PENDAHULUAN	4
1.1 Latar Belakang	4
1.2 Tujuan Praktikum	4
1.3 Manfaat	4
BAB II PEMBAHASAN	5
2.1 Pengertian Double Linked List	5
2.2 Operasi Dasar Double Linked List	5
2.3 Implementasi Dalam Java	6
2.3.1 NodeDLL_2511533031	6
2.3.2 InsertDLL_2511533031	7
2.3.3 PenelusuranDLL_2511533031	8
2.3.4 HapusDLL_2511533031	10
BAB III PENUTUP	12
3.1 Kesimpulan	12
DAFTAR PUSTAKA	13
LAPORAN TUGAS	14

BAB I

PENDAHULUAN

1.1 Latar Belakang

Salah satu struktur data yang sering digunakan adalah Linked List, yaitu kumpulan node yang saling terhubung menggunakan pointer. Pada praktikum sebelumnya telah dipelajari Single Linked List yang hanya memiliki satu pointer menuju node berikutnya. Namun, struktur tersebut memiliki keterbatasan karena penelusuran data hanya dapat dilakukan dalam satu arah.

Untuk mengatasi keterbatasan tersebut, digunakan Double Linked List yang memiliki dua pointer pada setiap node, yaitu pointer ke node berikutnya dan pointer ke node sebelumnya. Dengan adanya dua pointer tersebut, proses penelusuran, penambahan, maupun penghapusan data dapat dilakukan dengan lebih fleksibel. Oleh karena itu, praktikum ini dilakukan agar mahasiswa dapat memahami konsep.

1.2 Tujuan Praktikum

Adapun tujuan dari praktikum pekan ke-6 ini adalah sebagai berikut:

1. Memahami konsep dasar struktur data *Double Linked List*.
2. Mengetahui cara membuat dan menghubungkan node pada *Double Linked List*.
3. Memahami proses penambahan, penghapusan, pencarian, dan penelusuran data pada *Double Linked List*.

1.3 Manfaat

Manfaat yang diperoleh dari praktikum ini adalah sebagai berikut:

1. Mahasiswa dapat memahami cara kerja struktur data *Double Linked List*.
2. Mahasiswa dapat memahami cara kerja struktur data *Double Linked List*.
3. Mahasiswa dapat memahami penggunaan pointer sebelumnya (*prev*) dan pointer berikutnya (*next*) dalam pengelolaan data.

BAB II

PEMBAHASAN

2.1 Pengertian Double Linked List

Double Linked List merupakan struktur data linear yang terdiri dari kumpulan node yang saling terhubung. Setiap node pada *Double Linked List* memiliki tiga bagian, yaitu data, pointer ke node berikutnya (*next*), dan pointer ke node sebelumnya (*prev*). Dengan adanya dua pointer tersebut, proses penelusuran data dapat dilakukan dari depan ke belakang maupun dari belakang ke depan.

Berbeda dengan *Single Linked List* yang hanya memiliki satu arah penelusuran, *Double Linked List* memberikan fleksibilitas yang lebih baik dalam operasi penambahan dan penghapusan data. Struktur data ini sering digunakan pada aplikasi yang membutuhkan navigasi dua arah, seperti fitur undo-redo, playlist musik, dan navigasi halaman.

2.2 Operasi Dasar Double Linked List

Beberapa operasi dasar yang dapat dilakukan pada *Double Linked List* antara lain:

- 1. Traversal**

Digunakan untuk menelusuri dan menampilkan seluruh data dalam linked list, baik dari depan ke belakang maupun sebaliknya.

- 2. Insertion**

Digunakan untuk menambahkan node baru, baik di awal, akhir, maupun posisi tertentu dalam linked list.

- 3. Deletion**

Digunakan untuk menghapus node dari linked list, baik node pertama, node terakhir, maupun node pada posisi tertentu.

4. Searching

Digunakan untuk mencari data tertentu di dalam linked list dengan menelusuri node satu per satu.

2.3 Implementasi Dalam Java

Pada praktikum pekan ke-5 telah dibuat beberapa kode program yang berkaitan dengan implementasi struktur data **Double LinkedList** dalam bahasa pemrograman Java.

2.3.1 NodeDLL_2511533031

```
1 package pekan6_2511533031;
2
3 public class NodeDLL_2511533031 {
4     // mendefinisikan kelas node
5     int data_3031; // data
6     NodeDLL_2511533031 next_3031; // pointer ke next node
7     NodeDLL_2511533031 prev_3031; // pointer ke previous node
8
9     //konstruktor
10    public NodeDLL_2511533031(int data_3031) {
11        this.data_3031 = data_3031;
12        this.next_3031 = null;
13        this.prev_3031 = null;
14    }
15 }
```

Gambar 1

Class NodeDLL digunakan untuk membuat node pada struktur data *Double Linked List*. Class ini memiliki atribut data untuk menyimpan nilai, next untuk menunjuk node berikutnya, dan prev untuk menunjuk node sebelumnya. Konstruktor digunakan untuk menginisialisasi data node dan mengatur pointer next serta prev menjadi null.

2.3.2 InsertDLL_2511533031

```
1 package pekan6_2511533031;
2
3 public class InsertDLL_2511533031 {
4     //menambahkan node di awal DLL
5     static NodeDLL_2511533031 insertBegin(NodeDLL_2511533031 head_3031, int data_3031) {
6         // buat node baru
7         NodeDLL_2511533031 new_node_3031 = new NodeDLL_2511533031(data_3031);
8         // jadikan pointer nextnya head
9         new_node_3031.next_3031 = head_3031;
10        // jadikan pointer prev head ke new_node
11        if (head_3031 != null) {
12            head_3031.prev_3031 = new_node_3031;
13        }
14        return new_node_3031;
15    }
16    // fungsi menambahkan node di akhir
17    public static NodeDLL_2511533031 insertEnd(NodeDLL_2511533031 head_3031, int newData_3031) {
18        //buat node baru
19        NodeDLL_2511533031 new_node_3031 = new NodeDLL_2511533031(newData_3031);
20        // jika DLL null jadikan end
21        if (head_3031 == null) {
22            head_3031 = new_node_3031;
23        }
24        else {
25            NodeDLL_2511533031 curr_3031 = head_3031;
26            while (curr_3031.next_3031 != null) {
27                curr_3031 = curr_3031.next_3031;
28            }
29            curr_3031.next_3031 = new_node_3031;
30            new_node_3031.prev_3031 = curr_3031;
31        }
32        return head_3031;
33    }
34    // fungsi menambahkan node di posisi tertentu
35    public static NodeDLL_2511533031 insertAtPosition(NodeDLL_2511533031 head_3031, int pos_3031, int new_data_3031) {
36        // buat node baru
37        NodeDLL_2511533031 new_node_3031 = new NodeDLL_2511533031(new_data_3031);
38        if (pos_3031 == 1) {
39            new_node_3031.next_3031 = head_3031;
40            if (head_3031 != null) {
41                head_3031.prev_3031 = new_node_3031;
42            }
43            head_3031 = new_node_3031;
44            return head_3031;
45        }
46        NodeDLL_2511533031 curr_3031 = head_3031;
47        for (int i = 1; i < pos_3031 - 1 && curr_3031 != null; ++i) {
48            curr_3031 = curr_3031.next_3031;
49        }
50        if (curr_3031 == null) {
51            System.out.println("Posisi tidak ada");
52            return head_3031;
53        }
54        new_node_3031.prev_3031 = curr_3031;
55        new_node_3031.next_3031 = curr_3031.next_3031;
56        curr_3031.next_3031 = new_node_3031;
57        if (new_node_3031.next_3031 != null) {
58            new_node_3031.next_3031.prev_3031 = new_node_3031;
59        }
60        return head_3031;
61    }
62    public static void printList(NodeDLL_2511533031 head_3031) {
63        NodeDLL_2511533031 curr_3031 = head_3031;
64        while (curr_3031 != null) {
65            System.out.print(curr_3031.data_3031 + " <-> ");
66            curr_3031 = curr_3031.next_3031;
67        }
68        System.out.println();
69    }
70    public static void main(String[] args) {
71        // membuat dll 2 <-> 3 <-> 5
72        NodeDLL_2511533031 head_3031 = new NodeDLL_2511533031(2);
73        head_3031.next_3031 = new NodeDLL_2511533031(3);
74        head_3031.next_3031.prev_3031 = head_3031;
75        head_3031.next_3031.next_3031 = new NodeDLL_2511533031(5);
76        head_3031.next_3031.next_3031.prev_3031 = head_3031.next_3031;
77
78        //cetak DLL awal
79        System.out.print("DLL Awal: ");
80        printList(head_3031);
81
82        // tambah 1 di awal
83        head_3031 = insertBegin(head_3031, 1);
84        System.out.print(
85            "simpul 1 ditambah di awal: ");
86        printList(head_3031);
87
88        // tambah 6 di akhir
89        System.out.print(
90            "simpul 6 ditambah di akhir: ");
91        int data_3031 = 6;
92        head_3031 = insertEnd(head_3031, data_3031);
93        printList(head_3031);
94
95        // menambah node 4 di posisi 4
96        System.out.print("tambah node 4 di posisi 4: ");
97        int data2_3031 = 4;
98        int pos_3031 = 4;
99        head_3031 = insertAtPosition(head_3031, pos_3031, data2_3031);
100        printList(head_3031);
101    }
102 }
```

Gambar 2

Class InsertDLL digunakan untuk menambahkan node pada struktur data *Double Linked List*. Program ini menyediakan method insertBegin()

untuk menambahkan node di awal list, insertEnd() untuk menambahkan node di akhir list, dan insertAtPosition() untuk menambahkan node pada posisi tertentu. Pada setiap proses penambahan, pointer next dan prev diperbarui agar hubungan antar node tetap terhubung dengan benar.

Selain itu, terdapat method printList() yang digunakan untuk menampilkan isi *Double Linked List* dari awal hingga akhir. Pada method main, dibuat beberapa node awal kemudian dilakukan proses penambahan node di awal, akhir, dan posisi tertentu untuk menunjukkan cara kerja operasi insert pada *Double Linked List*.

2.3.3 PenelusuranDLL_2511533031

```
1 package pekang_2511533031;
2 public class PenelusuranDLL_2511533031 {
3     // fungsi penelusuran maju
4     static void forwardTraversal(NodeDLL_2511533031 head_3031) {
5         // memulai penelusuran dari head
6         NodeDLL_2511533031 curr_3031 = head_3031;
7         // lanjutkan sampai akhir
8         while (curr_3031 != null) {
9             // print data
10            System.out.print(curr_3031.data_3031 + " <-> ");
11            // pindah ke node berikutnya
12            curr_3031 = curr_3031.next_3031;
13        }
14        // print spasi
15        System.out.println();
16    }
17    // fungsi penelusuran mundur
18    static void backwardTraversal(NodeDLL_2511533031 tail_3031) {
19        // mulai dari akhir
20        NodeDLL_2511533031 curr_3031 = tail_3031;
21        // lanjut sampai head
22        while (curr_3031 != null) {
23            // cetak data
24            System.out.print(curr_3031.data_3031 + " <-> ");
25            // pindah ke node sebelumnya
26            curr_3031 = curr_3031.prev_3031;
27        }
28        // cetak spasi
29        System.out.println();
30    }
31    public static void main(String[] args) {
32        // cetak DLL
33        NodeDLL_2511533031 head_3031 = new NodeDLL_2511533031(1);
34        NodeDLL_2511533031 second_3031 = new NodeDLL_2511533031(2);
35        NodeDLL_2511533031 third_3031 = new NodeDLL_2511533031(3);
36
37        head_3031.next_3031 = second_3031;
38        second_3031.prev_3031 = head_3031;
39        second_3031.next_3031 = third_3031;
40        third_3031.prev_3031 = second_3031;
41
42        System.out.println("Penelusuran maju: ");
43        forwardTraversal(head_3031);
44
45        System.out.println("Penelusuran Mundur: ");
46        backwardTraversal(third_3031);
47    }
48 }
```

Gambar 3

Class PenelusuranDLL digunakan untuk melakukan penelusuran pada struktur data *Double Linked List*. Method forwardTraversal() berfungsi menampilkan data dari depan ke belakang menggunakan pointer

next, sedangkan method `backwardTraversal()` digunakan untuk menampilkan data dari belakang ke depan menggunakan pointer `prev`.

Pada method `main`, dibuat beberapa node yang saling terhubung membentuk *Double Linked List*. Setelah itu dilakukan penelusuran maju dan mundur untuk menunjukkan bahwa *Double Linked List* dapat diakses dalam dua arah.

2.3.4 HapusDLL_2511533031

```
1 package pekan6_2511533031;
2
3 public class HapusDLL_2511533031 {
4     // fungsi menghapus node awal
5     public static NodeDLL_2511533031 delHead(NodeDLL_2511533031 head_3031) {
6         if (head_3031 == null) {
7             return null;
8         }
9         NodeDLL_2511533031 temp_3031 = head_3031;
10        head_3031 = head_3031.next_3031;
11        if (head_3031 != null) {
12            head_3031.prev_3031 = null;
13        }
14        return head_3031;
15    }
16    // fungsi menghapus di akhir
17    public static NodeDLL_2511533031 dellast(NodeDLL_2511533031 head_3031) {
18        if (head_3031 == null) {
19            return null;
20        }
21        if (head_3031.next_3031 == null) {
22            return null;
23        }
24        NodeDLL_2511533031 curr_3031 = head_3031;
25        while (curr_3031.next_3031 != null) {
26            curr_3031 = curr_3031.next_3031;
27        }
28        // update pointer previous node
29        if (curr_3031.prev_3031 != null) {
30            curr_3031.prev_3031.next_3031 = null;
31        }
32        return head_3031;
33    }
34    // fungsi menghapus node posisi tertentu
35    public static NodeDLL_2511533031 delPos(NodeDLL_2511533031 head_3031, int pos_3031) {
36        // jika DLL kosong
37        if (head_3031 == null) {
38            return head_3031;
39        }
40        NodeDLL_2511533031 curr_3031 = head_3031;
41        // telusuri sampai ke node yang akan dihapus
42        for (int i_3031 = 1; curr_3031 != null && i_3031 < pos_3031; ++i_3031) {
43            curr_3031 = curr_3031.next_3031;
44        }
45        // jika posisi tidak ditemukan
46        if (curr_3031 == null) {
47            return head_3031;
48        }
49        // Update pointer
50        if (curr_3031.prev_3031 != null) {
51            curr_3031.prev_3031.next_3031 = curr_3031.next_3031;
52        }
53        if (curr_3031.next_3031 != null) {
54            curr_3031.next_3031.prev_3031 = curr_3031.prev_3031;
55        }
56        // jika yang dihapus head
57        if (head_3031 == curr_3031) {
58            head_3031 = curr_3031.next_3031;
59        }
60        return head_3031;
61    }
62    // fungsi mencetak DLL
63    public static void printList(NodeDLL_2511533031 head_3031) {
64        NodeDLL_2511533031 curr_3031 = head_3031;
65        while (curr_3031 != null) {
66            System.out.print(curr_3031.data_3031 + " ");
67            curr_3031 = curr_3031.next_3031;
68        }
69        System.out.println();
70    }
71    public static void main(String[] args) {
72        // buat sebuah DLL
73        NodeDLL_2511533031 head_3031 = new NodeDLL_2511533031(1);
74        head_3031.next_3031 = new NodeDLL_2511533031(2);
75        head_3031.next_3031.prev_3031 = head_3031;
76        head_3031.next_3031.next_3031 = new NodeDLL_2511533031(3);
77        head_3031.next_3031.next_3031.prev_3031 = head_3031.next_3031;
78        head_3031.next_3031.next_3031.next_3031 = new NodeDLL_2511533031(4);
79        head_3031.next_3031.next_3031.next_3031.prev_3031 = head_3031.next_3031.next_3031;
80        head_3031.next_3031.next_3031.next_3031.next_3031 = new NodeDLL_2511533031(5);
81        head_3031.next_3031.next_3031.next_3031.next_3031.prev_3031 = head_3031.next_3031.next_3031.next_3031;
82
83        System.out.print("DLL Awal: ");
84        printList(head_3031);
85
86        System.out.print("Setelah head dihapus: ");
87        head_3031 = delHead(head_3031);
88        printList(head_3031);
89
90        System.out.print("Setelah node terakhir dihapus: ");
91        head_3031 = dellast(head_3031);
92        printList(head_3031);
93
94        System.out.print("Menghapus node ke 2: ");
95        head_3031 = delPos(head_3031, 2);
96        printList(head_3031);
97    }
98 }
```

Gambar 4

Class HapusDLL digunakan untuk menghapus node pada struktur data *Double Linked List*. Program ini memiliki beberapa method, yaitu delHead() untuk menghapus node pertama, delLast() untuk menghapus node terakhir, dan delPos() untuk menghapus node pada posisi tertentu. Pada setiap proses penghapusan, pointer next dan prev diperbarui agar hubungan antar node tetap terhubung dengan benar setelah node dihapus.

Selain itu, terdapat method printList() yang berfungsi menampilkan isi *Double Linked List* dari awal hingga akhir. Pada method main, program membuat beberapa node yang saling terhubung, kemudian melakukan proses penghapusan pada bagian awal, akhir, dan posisi tertentu untuk memperlihatkan perubahan isi linked list setelah operasi delete dilakukan.

BAB III

PENUTUP

3.1 Kesimpulan

Pada praktikum pekan ke-6 ini dapat dipahami bahwa *Double Linked List* merupakan struktur data linear yang setiap node-nya memiliki dua pointer, yaitu next dan prev, sehingga data dapat ditelusuri dalam dua arah. Dengan struktur tersebut, proses penambahan, penghapusan, dan penelusuran data menjadi lebih fleksibel dibandingkan *Single Linked List*.

Melalui praktikum ini juga dipelajari implementasi berbagai operasi dasar pada *Double Linked List*, seperti insert, traversal, dan delete menggunakan bahasa pemrograman Java. Dari hasil praktikum dapat disimpulkan bahwa penggunaan pointer prev dan next sangat penting untuk menjaga hubungan antar node agar struktur linked list tetap berjalan dengan baik.

DAFTAR PUSTAKA

- [1] GeeksforGeeks, "Doubly Linked List Tutorial," GeeksforGeeks, 19 September 2025. [Online]. Available: <https://www.geeksforgeeks.org/dsa/doubly-linked-list/>. [Accessed 13 Mei 2026].

- [2] Programiz, "Doubly Linked List Data Structure," Programiz, [Online]. Available: https://www.programiz.com/dsa/doubly-linked-list?utm_source=chatgpt.com. [Accessed 13 Mei 2026].

LAPORAN TUGAS

A. Kode Program

Berikut kode program yang telah dibuat pada tugas praktikum pekan-6:

1. Lagu_2511533031

```
1 package pekan6_2511533031;
2
3 public class Lagu_2511533031 {
4
5     // Atribut data lagu
6     private String judul_3031;
7     private String penyanyi_3031;
8
9     // Pointer ke node berikutnya dan sebelumnya
10    Lagu_2511533031 next_3031;
11    Lagu_2511533031 prev_3031;
12
13    // Constructor
14    public Lagu_2511533031(String judul_3031, String penyanyi_3031) {
15        this.judul_3031 = judul_3031;
16        this.penyanyi_3031 = penyanyi_3031;
17        this.next_3031 = null;
18        this.prev_3031 = null;
19    }
20
21    // Getter
22    public String getJudul_3031() {
23        return judul_3031;
24    }
25
26    public String getPenyanyi_3031() {
27        return penyanyi_3031;
28    }
29
30    // Setter
31    public void setJudul_3031(String judul_3031) {
32        this.judul_3031 = judul_3031;
33    }
34
35    public void setPenyanyi_3031(String penyanyi_3031) {
36        this.penyanyi_3031 = penyanyi_3031;
37    }
38 }
```

Gambar 5

Kelas **Lagu_2511533031** adalah representasi satu node dalam Double Linked List. Setiap objek menyimpan dua data yaitu judul_3031 dan penyanyi_3031, serta dua pointer next_3031 dan prev_3031 yang masing-masing menunjuk ke lagu berikutnya dan sebelumnya. Saat constructor dipanggil, data lagu langsung diisi sedangkan kedua pointer diset null karena belum terhubung ke node manapun. Akses ke atribut private dilakukan secara terkontrol melalui getter dan setter.

2. Musik_2511533031

```
1. package pekan6_2511533031;
2.
3. import java.util.*;
4. public class Musik_2511533031 {
5.     // Pointer kepala (awal) dan ekor (akhir) DLL
6.     private Lagu_2511533031 head_3031;
7.     private Lagu_2511533031 tail_3031;
8.
9.     // Constructor - playlist dimulai kosong
```

```

10.     public Musik_2511533031() {
11.         head_3031 = null;
12.         tail_3031 = null;
13.     }
14.     // 1. tambahLagu_3031 - menambah lagu baru di AKHIR
    playlist
15.     public void tambahLagu_3031(String judul, String
    penyanyi) {
16.         Lagu_2511533031 lagubaru_3031 = new
    Lagu_2511533031(judul, penyanyi);
17.
18.         if (head_3031 == null) {
19.             // Playlist kosong - node pertama sekaligus jadi
    head dan tail
20.             head_3031 = lagubaru_3031;
21.             tail_3031 = lagubaru_3031;
22.         } else {
23.             // Sambungkan ke akhir list
24.             lagubaru_3031.prev_3031 = tail_3031;
25.             tail_3031.next_3031     = lagubaru_3031;
26.             tail_3031                 = lagubaru_3031;
27.         }
28.         System.out.println("Lagu berhasil ditambahkan!");
29.     }
30.
31.     // 2. hapusLaguAwal_3031 - menghapus lagu pertama (head)
32.     public void hapusLaguAwal_3031() {
33.         if (head_3031 == null) {
34.             System.out.println("Playlist kosong, tidak ada
    lagu yang dihapus.");
35.             return;
36.         }
37.
38.         System.out.println("Menghapus lagu: \"\" +
    head_3031.getJudul_3031() + "\" - \"\" +
    head_3031.getPenyanyi_3031());
39.
40.         if (head_3031 == tail_3031) {
41.             // Hanya ada satu lagu, kosongkan list
42.             head_3031 = null;
43.             tail_3031 = null;
44.         } else {
45.             // Geser head ke node berikutnya, putus referensi
    lama
46.             head_3031             = head_3031.next_3031;
47.             head_3031.prev_3031 = null;
48.         }
49.         System.out.println("Lagu berhasil dihapus!");
50.     }
51.     // 3. tampilMaju_3031 - menampilkan semua lagu awal →
    akhir
52.     public void tampilMaju_3031() {
53.         if (head_3031 == null) {
54.             System.out.println("Playlist kosong.");

```

```

55.         return;
56.     }
57.
58.     System.out.println("==== Playlist (Maju) =====");
59.     Lagu_2511533031 current_3031 = head_3031;
60.     int nomor_3031 = 1;
61.
62.     while (current_3031 != null) {
63.         System.out.println(nomor_3031 + ". " +
64.             current_3031.getJudul_3031()
65.                 + " - " +
66.             current_3031.getPenyanyi_3031());
67.         current_3031 = current_3031.next_3031;
68.         nomor_3031++;
69.     }
70.     System.out.println("=====");
71. // -----
72. // 4. tampilMundur_3031 - menampilkan semua lagu akhir →
73. // awal
74. // (memanfaatkan pointer prev - fitur utama DLL)
75. // -----
76. public void tampilMundur_3031() {
77.     if (tail_3031 == null) {
78.         System.out.println("Playlist kosong.");
79.         return;
80.     }
81.     System.out.println("==== Playlist (Mundur) =====");
82.     Lagu_2511533031 current_3031 = tail_3031;
83.     int nomor_3031 = 1;
84.
85.     while (current_3031 != null) {
86.         System.out.println(nomor_3031 + ". " +
87.             current_3031.getJudul_3031()
88.                 + " - " +
89.             current_3031.getPenyanyi_3031());
90.         current_3031 = current_3031.prev_3031; //
91.         mundur lewat pointer prev
92.         nomor_3031++;
93.     }
94.     System.out.println("=====");
95. }
96. // 5. cariLagu_3031 - mencari lagu berdasarkan judul
97. // (tidak case-sensitive)
98. public void cariLagu_3031(String judul) {
99.     if (head_3031 == null) {
100.         System.out.println("Playlist kosong, tidak ada
101.         lagu untuk dicari.");
102.         return;
103.     }

```



```

100.
101.     Lagu_2511533031 current_3031 = head_3031;
102.     int posisi_3031 = 1;
103.     boolean ditemukan_3031 = false;
104.
105.     while (current_3031 != null) {
106.         if
107.         (current_3031.getJudul_3031().equalsIgnoreCase(judul)) {
108.             System.out.println("Lagu ditemukan di
109.             posisi " + posisi_3031 + ":");
110.             System.out.println("  Judul      : " +
111.             current_3031.getJudul_3031());
112.             System.out.println("  Penyanyi : " +
113.             current_3031.getPenyanyi_3031());
114.             ditemukan_3031 = true;
115.             break;
116.         }
117.         current_3031 = current_3031.next_3031;
118.         posisi_3031++;
119.     }
120.
121.     if (!ditemukan_3031) {
122.         System.out.println("Lagu \"" + judul + "\"
123.         tidak ditemukan dalam playlist.");
124.     }
125. }
126. // main - program utama dengan menu interaktif
127. public static void main(String[] args) {
128.     Scanner scanner_3031 = new Scanner(System.in);
129.     Musik_2511533031 playlist_3031 = new
130.     Musik_2511533031();
131.
132.     int pilihan_3031;
133.
134.     do {
135.         System.out.println("=== Playlist Musik NIM:
136.         2511533031 ===");
137.         System.out.println("1. Tambah Lagu");
138.         System.out.println("2. Hapus Lagu Pertama");
139.         System.out.println("3. Lihat Playlist (Maju)");
140.         System.out.println("4. Lihat Playlist
141.         (Mundur)");
142.         System.out.println("5. Cari Lagu");
143.         System.out.println("6. Keluar");
144.         System.out.print("Pilihan: ");
145.
146.         pilihan_3031 = scanner_3031.nextInt();
147.         scanner_3031.nextLine(); // buang newline sisa
148.
149.         switch (pilihan_3031) {
150.             case 1:
151.                 System.out.print("Judul      : ");
152.                 String judul_3031 =
153.                 scanner_3031.nextLine();

```

```

145.         System.out.print("Penyanyi: ");
146.         String penyanyi_3031 =
            scanner_3031.nextLine();
147.         playlist_3031.tambahLagu_3031(judul_3031, penyanyi_3031);
148.         break;
149.         case 2:
150.             playlist_3031.hapusLaguAwal_3031();
151.             break;
152.         case 3:
153.             playlist_3031.tampilMaju_3031();
154.             break;
155.         case 4:
156.             playlist_3031.tampilMundur_3031();
157.             break;
158.         case 5:
159.             System.out.print("Masukkan judul lagu
yang dicari: ");
160.             String cari_3031 =
                scanner_3031.nextLine();
161.             playlist_3031.cariLagu_3031(cari_3031);
162.             break;
163.         case 6:
164.             System.out.println("Terima kasih!
Program selesai.");
165.             break;
166.         default:
167.             System.out.println("Pilihan tidak
valid. Silakan coba lagi.");
168.     }
169.
170.     } while (pilihan_3031 != 6);
171.
172.     scanner_3031.close();
173. }
174. }

```

Kelas Musik_2511533031 adalah kelas utama yang mengelola playlist berbasis Double Linked List menggunakan dua pointer head_3031 (node pertama) dan tail_3031 (node terakhir), keduanya diset null saat playlist masih kosong.

Kelas ini memiliki lima operasi utama. tambahLagu_3031() membuat node baru dan menyambungkannya di akhir list dengan menghubungkan pointer prev dan next antara tail lama dan node baru. hapusLaguAwal_3031() menghapus head dengan menggeser head ke node berikutnya lalu memutus pointer prev-nya. tampilMaju_3031() menelusuri list dari head ke tail lewat pointer next_3031, sedangkan

tampilMundur_3031() menelusuri dari tail ke head lewat pointer prev_3031 — inilah fitur utama DLL yang tidak bisa dilakukan Single Linked List. cariLagu_3031() menelusuri list dari awal dan membandingkan judul secara case-insensitive menggunakan equalsIgnoreCase(). Semua operasi dilengkapi pengecekan kondisi kosong agar program tidak error.

Terakhir, method main() menjalankan program dengan menu interaktif berbasis Scanner dan loop do-while yang terus berjalan hingga pengguna memilih opsi keluar.

B. Output Program

Berikut output dari kode program yang telah dibuat.

1.

```
=== Playlist Musik NIM: 2511533031 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul   : Soulmate
Penyanyi: Kahitna
Lagu berhasil ditambahkan!
```
2.

```
=== Playlist Musik NIM: 2511533031 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3
===== Playlist (Maju) =====
1. Soulmate - Kahitna
2. Goodbye - Air Suply
=====
```
3.

```
=== Playlist Musik NIM: 2511533031 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 4
===== Playlist (Mundur) =====
1. Goodbye - Air Suply
2. Soulmate - Kahitna
=====
```

```
=== Playlist Musik NIM: 2511533031 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 5
Masukkan judul lagu yang dicari: soulmate
Lagu ditemukan di posisi 1:
    Judul      : Soulmate
    Penyanyi   : Kahitna
```

4.

```
=== Playlist Musik NIM: 2511533031 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 2
Menghapus lagu: "Soulmate" - Kahitna
Lagu berhasil dihapus!
```

5.